

Proof Calculi for Propositional Logic

Kumar Shashwat

September 29, 2026

Abstract

This paper formalizes propositional logic as a 4-tuple system and examines two proof calculi - Resolution and Hilbert calculus with respect to soundness, completeness and their suitability for automating derivation of a *target* from a set of assumptions. Resolution is used for proofs by refutation and requires pre-processing to obtain a *refutation set* of clauses, while Hilbert calculus derives formulas directly using three axiom schemas and Modus Ponens and requires pre-processing to eliminate conjunctions and disjunctions. The two proof calculi are seen in action by proving reachability in a directed graph by encoding the reachability question as a propositional formula.

1 Introduction

A mathematical proof is a sequence of *logical* steps that establishes a certain *fact* given a premise (set of assumptions i.e. statements that are assumed to be true). A proof calculus is a set of *rules* that can be used to establish a certain fact given a premise (set of assumptions) within a *logical system*. In this paper, we study two different proof systems for propositional logic. We start with defining propositional logic system first and see how different proof calculi can be used within the system. The goal of the report is to define the terms in a way that definitions can serve as a recipe for automating proofs. By treating proofs as a mathematical object, we can study whether we can prove a fact from a certain proof calculus and it also makes it easier to automate proving facts. In the background section, we give an overview about terminologies that we use throughout the document. We also formally define the propositional logic system and how proof calculus fits into it. After that, we look at Resolution calculus and Hilbert calculus and their properties and assess their suitability with respect to automation. We also look at an example of applying proof calculus to prove reachability in a directed graph.

2 Background

Before presenting the propositional logic system, we introduce several terminologies that will be used throughout this text (Gallier [1986]).

1. **Well-Formed Formulas:** A *well-formed formula* (or simply *formula*) is an expression built from atomic symbols that is syntactically valid according to the rules of a logic system. .
2. **Partial Function:** A *partial function* $f : A \rightarrow B$ is a function that may not be defined on every element of its domain A . We write $f(a) = b$ when f is defined at a and returns b , and say f is *undefined* at a otherwise. Partial functions arise naturally in the definition of inference rules (Definition 3.2).
3. **Premise and Target:** The *premise* is a set of facts that we know to hold while *target* is the conclusion that we want to *derive* given the premise holds.

3 The Propositional Logic System

Before we go into what constitutes a logical step or a proof or a proof calculus, we need to formally define a propositional logic system (Enderton [2001], Wasilewska [2018]).

Definition 3.1 (Propositional Logic System). A propositional logic system is a 4-tuple $\mathcal{L} = \langle \Sigma, \mathcal{F}, M, D \rangle$ comprising:

1. Σ is a finite set of atomic symbols, i.e. symbols that cannot be decomposed in terms of other symbols. $\Sigma = V \cup C \cup P$, where $V = \{p_1, p_2, p_3, \dots, p_n\}$ is a finite set of propositional variables, $C = \{\neg, \wedge, \vee, \rightarrow, \top, \perp\}$ is the set of logical connectives, and $P = \{(\,,\,)\}$ is the set of punctuation symbols. The set of all strings (or words) generated from Σ is denoted as Σ^* .
2. $\mathcal{F}$ is a *decidable* subset (i.e. we can check in finite time if a word is in the set) $\mathcal{F} \subset \Sigma^*$ consisting of *well-formed formulas*. The set $\mathcal{F}$ is generated by the following inductive rules.
 - Base Cases: The following are formulas in $\mathcal{F}$:
 - (F1) Any propositional variable $p \in V$
 - (F2) $\top$
 - (F3) $\perp$
 - Inductive Steps: If ϕ and ψ are formulas in $\mathcal{F}$, the following are also formulas in $\mathcal{F}$:
 - (F4) $\neg\phi$
 - (F5) $(\phi \wedge \psi)$
 - (F6) $(\phi \vee \psi)$
 - (F7) $(\phi \rightarrow \psi)$

Inductive Steps. If ϕ and ψ are formulas in $\mathcal{F}$, the following are also formulas in $\mathcal{F}$: (F4) $\neg\phi$; (F5) $(\phi \wedge \psi)$; (F6) $(\phi \vee \psi)$; (F7) $(\phi \rightarrow \psi)$.

3. M is the tuple $\langle \mathcal{I}, \models \rangle$. Here, $\mathcal{I}$ is a non-empty set of interpretations, which consists of all total boolean valuation functions $v : V \rightarrow \{0, 1\}$, i.e. mapping every propositional variable to a boolean truth value.

The symbol $\models$ denotes the *satisfaction relation*, which is a binary relation defined as a subset $\models \subseteq \mathcal{I} \times \mathcal{F}$. We say that a formula ϕ holds(or is satisfied) under an interpretation v (written $v \models \phi$) based on structural induction over the syntax of ϕ . We write $v \not\models \phi$ to assert that a formula does not hold under an interpretation. The inductive rules are:

- Base Cases:

$$(S1) \quad v \models p_i \quad \text{iff} \quad v(p_i) = 1 \quad (\text{for any } p_i \in V)$$

$$(S2) \quad v \models \top \text{ always holds} \quad (\text{i.e., } (v, \top) \in \models)$$

$$(S3) \quad v \not\models \perp \text{ always holds} \quad (\text{i.e., } (v, \perp) \notin \models)$$

- Inductive Steps:

$$(S4) \quad v \models \neg\phi \quad \text{iff} \quad v \not\models \phi$$

$$(S5) \quad v \models (\phi \wedge \psi) \quad \text{iff} \quad v \models \phi \text{ and } v \models \psi$$

$$(S6) \quad v \models (\phi \vee \psi) \quad \text{iff} \quad v \models \phi \text{ or } v \models \psi$$

$$(S7) \quad v \models (\phi \rightarrow \psi) \quad \text{iff} \quad v \not\models \phi \text{ or } v \models \psi$$

4. D is a selectable component of a logic system i.e. for a fixed $\langle \Sigma, \mathcal{F}, M \rangle$, we may choose from different D that can help us derive formulas. Formally, $D = \langle \Sigma_A, R \rangle$, where Σ_A is a finite set of *axiom schemas* and R is a finite set of computable inference rules (see Definitions 3.2 and 3.3).

In a system $\mathcal{L} = \langle \Sigma, \mathcal{F}, M, D \rangle$, we refer to Σ as the *alphabet* of $\mathcal{L}$, $\mathcal{F}$ as the *language*, M as the *semantics*, and D as the *proof calculus*.

In English, a *proposition* is a sentence that can either be true or false. In interpretations, 0 stands for false and 1 stands for true. These boolean valuation functions in $\mathcal{I}$ provide meaning to the formulas by assigning a truth value to each propositional variable. Propositional logic aims at formalizing the reasoning over propositions.

Definition 3.2 (Inference Rule). An inference rule r is a computable partial function:

$$f_r : \mathcal{P}(\mathcal{F}) \rightharpoonup \mathcal{F}$$

that maps a finite set of input formulas called the *premise* to a single output formula. They are partial functions because an inference rule cannot always be applied to any arbitrary set of formulas. $\mathcal{P}(\mathcal{F})$ denotes the power set of $\mathcal{F}$, i.e. the set of all subsets of formulas.

If $f_r(\{\psi_1, \dots, \psi_k\}) = \phi$, we adopt the convention of writing:

$$\frac{\psi_1 \quad \dots \quad \psi_k}{\phi}$$

Definition 3.3 (Axiom Schema). An *axiom schema* is a formula template σ in which *placeholder formulas* $\phi, \psi, \chi, \dots$ can be replaced with arbitrary formulas in $\mathcal{F}$ (Corcoran and Hamid [2022]). The *axiom set* $A(D)$ induced by an axiom schema set Σ_A is the set of all instances of all schemas in Σ_A :

$$A(D) = \{ \alpha \in \mathcal{F} \mid \exists \sigma \in \Sigma_A, \alpha \text{ is an instance of } \sigma \}$$

An axiom schema σ is *valid* if every one of its instances is satisfied by every interpretation $v \in \mathcal{I}$:

$$\forall \alpha \in A(D) \text{ s.t. } \alpha \text{ is an instance of } \sigma, \quad \forall v \in \mathcal{I}, v \models \alpha$$

One example of axiom schema would be $\phi \rightarrow (\psi \rightarrow \phi)$ where ϕ, ψ, χ range over all formulas in $\mathcal{F}$. For example, we can substitute ϕ, ψ, χ with p_1 . Then $p_1 \rightarrow (p_1 \rightarrow p_1)$ is an axiom generated from the aforementioned schema.

Definition 3.4 (Logical Consequence). We define *logical consequence* (denoted $\Gamma \models \phi$) as a binary relation between a set of premise formulas $\Gamma \subseteq \mathcal{F}$ and a conclusion formula $\phi \in \mathcal{F}$, meaning that for every interpretation $v \in \mathcal{I}$, if $v \models \gamma$ for all $\gamma \in \Gamma$, then $v \models \phi$. If this condition holds, we say ϕ is a *logical consequence* of Γ .

We reuse the satisfaction symbol $\models$ here in a new context. Previously, $\models$ was used as a relation between interpretations and formulas (the satisfaction relation in Definition 3.1). Here, we use it as a relation between *sets of formulas* and individual formulas to express logical consequence. In $\Gamma \models \phi$, we refer to Γ as the *premise set* and to ϕ as the *conclusion*.

Definition 3.5 (Logical Equivalence). Two formulas $\phi, \psi \in \mathcal{F}$ are *logically equivalent*, written $\phi \equiv \psi$, if for every interpretation $v \in \mathcal{I}$, either both ϕ and ψ are satisfied by v , or neither is. Formally:

$$\phi \equiv \psi \quad \text{iff} \quad \text{for all } v \in \mathcal{I}, (v \models \phi \text{ and } v \models \psi) \text{ or } (v \not\models \phi \text{ and } v \not\models \psi)$$

Equivalently, using Definition 3.4, $\phi \equiv \psi$ iff $\phi \models \psi$ and $\psi \models \phi$ (where $\phi \models \psi$ is shorthand notation for $\{\phi\} \models \psi$).

Definition 3.6 (Syntactic Derivability and Formal Proof). Let $T = \mathcal{P}(\mathcal{F})$ denote the set of all possible sets of premise formulas. For an arbitrarily chosen proof calculus $D = \langle \Sigma_A, R \rangle$, we define *syntactic derivability* as a binary relation $\vdash_D \subseteq T \times \mathcal{F}$. A *formal proof* from Γ is a finite sequence of formulas $\psi_1, \psi_2, \dots, \psi_n$ where for every $1 \leq i \leq n$, the formula ψ_i satisfies at least one of the following:

- (a) *Premise*: $\psi_i \in \Gamma$.
- (b) *Axiom Instance*: $\psi_i \in A(D)$, i.e. ψ_i is an instance of some schema in Σ_A (Definition 3.3).
- (c) *Rule Application*: There exists $r \in R$ with associated partial function f_r such that $\psi_i = f_r(\Gamma_{\text{in}})$ for some finite $\Gamma_{\text{in}} \subseteq \{\psi_1, \dots, \psi_{i-1}\}$.

We write $\Gamma \vdash_D \psi_n$ to denote that the pair $(\Gamma, \psi_n) \in \vdash_D$. We read it as we can derive ψ_n from the premise Γ using the calculus D . The formula ψ_n in the final step determines the type of derivation:

- (1) **Direct Derivation.** If $\psi_n = \phi$, the sequence is a direct proof of ϕ .
- (2) **Refutation.** For refutation, we assume the negation of the target formula to hold and obtain the logical contradiction in the last step. If $\psi_n = \perp$, the sequence is a *proof by refutation* of ϕ over the *augmented premise* set $\Gamma \cup \{\neg\phi\}$.

Definition 3.7 (Soundness). An inference rule is *sound* if its output formula is a logical consequence of its input premise. A proof calculus D is *sound* if all of its inference rules are sound and its axiom schemas are valid. Formally, D is sound if for any $\Gamma \subseteq \mathcal{F}$ and $\phi \in \mathcal{F}$:

$$\Gamma \vdash_D \phi \implies \Gamma \models \phi$$

Definition 3.8 (Completeness). A proof calculus D is *complete* if every logical consequence is syntactically derivable within that system. Formally, for any $\Gamma \subseteq \mathcal{F}$ and $\phi \in \mathcal{F}$:

$$\Gamma \models \phi \implies \Gamma \vdash_D \phi$$

There are some proof calculi that are not complete for direct derivation but complete for refutation. We call them *refutation-complete*. We are interested in proof calculi that are both sound and complete. Soundness guarantees that every derivable formula is a logical consequence of its premise, while completeness guarantees that every logical consequence can actually be derived (no valid statement is left unprovable). Together, they ensure that $\Gamma \vdash_D \phi$ holds *if and only if* $\Gamma \models \phi$.

Decidability with respect to the decision problem of finding a valid proof if there exists one and deciding that a proof cannot be derived when it doesn't exist is another important property when examining a proof calculus. However, it is not always the case that a decidable proof calculus exists for any arbitrary logic system. In first-order logic, Church theorem ensures that any proof calculus is undecidable (Fitting [1996]). As a result, first-order logic isn't very suitable for automation. By contrast, since we are working with propositional logic, a much simpler system, all these difficulties disappear. The set of propositional tautologies (formulas that are logical consequences of the empty premise set) is decidable (truth tables ensure decidability). This decidability is the key reason why the propositional logic system is suitable for proof automation.

4 Resolution Calculus

We will look into a proof calculus called the resolution calculus. While we can derive direct proofs for some valid formulas using resolution, the calculus is not complete for direct proofs. However, it is *refutation-complete* for formulas in the so-called *clausal* form (Gallier [1986]), meaning it can always derive refutation for formulas in that form. For this reason, we use proof by refutation : to prove $\Gamma \models \phi$, we add $\neg\phi$ to the premise and show that $\Gamma \cup \{\neg\phi\}$ is unsatisfiable. We derive $\square$ which is logically equivalent

to $\perp$ by repeatedly applying the *resolution rule* on the *clauses* in the *refutation set* constructed from the *augmented premise* $\Gamma \cup \{\neg\phi\}$. In order to construct the refutation set, we need to convert the formulas to *CNF*, then we need to construct a set of clauses from the CNF formulas, as the resolution rule takes a set of two clauses as input i.e. it only works on clauses.

4.1 Definitions

Definition 4.1 (Literal). Let V be the set of all propositional variables. We define a *literal* L is defined as either a variable itself (a *positive literal*) or the negation of a propositional variable (a *negative literal*). The set of all literals L is:

$$L = V \cup \{\neg p \mid p \in V\}$$

Definition 4.2 (Clause). A *clause* C is a finite disjunction (OR) of literals. A clause C_i containing m literals is written as:

$$C_i = (L_{i,1} \vee L_{i,2} \vee \cdots \vee L_{i,m})$$

A clause containing zero literals is called the *empty clause* (denoted $\square$). Because it represents a disjunction of zero literals, the empty clause is logically equivalent to a logical contradiction ($\square \equiv \perp$): by (S6) at least one of the literals needs to be satisfied, but since none are satisfied, it is equivalent to (S3) i.e. never satisfiable.

Definition 4.3 (Conjunctive Normal Form). We say that a formula ϕ is in *Conjunctive Normal Form* (CNF) if it is a finite conjunction (logical AND) of clauses. A CNF formula consisting of n clauses is written as:

$$\phi_{\text{CNF}} = (C_1 \wedge C_2 \wedge \cdots \wedge C_n)$$

Definition 4.4 (Logical Equivalences for CNF Transformation). In order to use the resolution calculus, all the formulas in the augmented premise must first be transformed into a logically equivalent formula ϕ_{CNF} (Definition 3.5). We can achieve this transformation by exhaustively applying the following logical equivalences:

1. *Implication Elimination*:

$$(\alpha \rightarrow \beta) \equiv (\neg\alpha \vee \beta)$$

2. *De Morgan's Laws*: For any sequence of $n \geq 2$ formulas:

$$\neg\left(\bigwedge_{i=1}^n \alpha_i\right) \equiv \left(\bigvee_{i=1}^n \neg\alpha_i\right) \quad \neg\left(\bigvee_{i=1}^n \alpha_i\right) \equiv \left(\bigwedge_{i=1}^n \neg\alpha_i\right)$$

3. *Double Negation Elimination*:

$$\neg\neg\alpha \equiv \alpha$$

4. Distributivity (Pushing $\vee$ s Inward):

$$\alpha \vee \left(\bigwedge_{i=1}^n \beta_i \right) \equiv \bigwedge_{i=1}^n (\alpha \vee \beta_i)$$

Definition 4.5 (Resolution Calculus). The *resolution calculus* is a proof calculus $D_{\text{res}} = \langle \Sigma_{A_{\text{res}}}, R_{\text{res}} \rangle$, where the axiom schema set is empty, i.e. $\Sigma_{A_{\text{res}}} = \emptyset$, and the rule set contains exactly one rule, $R_{\text{res}} = \{r_{\text{res}}\}$. The resolution rule defined in Definition 4.7.

Lemma 4.6 (Clause Extraction Equivalence). Let $\phi_{\text{CNF}} = C_1 \wedge C_2 \wedge \dots \wedge C_n$ be a formula in CNF with extracted clause set $S = \{C_1, \dots, C_n\}$. For any interpretation $v \in \mathcal{I}$:

$$v \models \phi_{\text{CNF}} \iff \forall C_i \in S, v \models C_i$$

i.e. we can replace a CNF formula with a set of its clauses as they are logically equivalent.

Proof. We proceed by induction on n , the number of clauses, using (S5) from Definition 3.1.

Base Case ($n = 1$): $\phi_{\text{CNF}} = C_1$ and $S = \{C_1\}$. Then $v \models C_1$ iff $\forall C_i \in \{C_1\}, v \models C_i$. The base case holds.

Induction Hypothesis: We assume that the lemma holds for some $k \geq 1$ clauses: $v \models (C_1 \wedge \dots \wedge C_k) \iff \forall C_i \in \{C_1, \dots, C_k\}, v \models C_i$.

Inductive Step ($n = k + 1$): By (S5),

$$v \models ((C_1 \wedge \dots \wedge C_k) \wedge C_{k+1}) \iff v \models (C_1 \wedge \dots \wedge C_k) \text{ and } v \models C_{k+1}$$

Applying the Inductive Hypothesis to the first conjunct:

$$\iff (\forall C_i \in \{C_1, \dots, C_k\}, v \models C_i) \text{ and } v \models C_{k+1} \iff \forall C_i \in S, v \models C_i$$

Hence the lemma holds by induction. $\square$

4.2 Resolution Refutation Procedure & Refutation Set Construction

Through proof by refutation, we show that $\neg\phi$ can never be a logical consequence of Γ . Given the premise $\Gamma \subseteq \mathcal{F}$ and a target formula $\phi \in \mathcal{F}$, we construct the *refutation set* S to make resolution rule applicable. First, we negate the target to obtain $\neg\phi$. We then apply the equivalence transformations in Definition 4.4 exhaustively to each $\gamma \in \Gamma$ and to $\neg\phi$, obtaining CNF formulas γ_{CNF} and $(\neg\phi)_{\text{CNF}}$ respectively. By Lemma 4.6, we can extract logically equivalent clause sets from each CNF formula. Collecting all such clauses and taking a union over them gives the refutation set $S = (\bigcup_{\gamma \in \Gamma} S(\gamma_{\text{CNF}})) \cup S((\neg\phi)_{\text{CNF}})$. By Lemma 4.6, any $v \in \mathcal{I}$ satisfies all formulas in $\Gamma \cup \{\neg\phi\}$ if and only if it satisfies every clause in S .

Definition 4.7 (Resolution Rule). The resolution rule r_{res} is an inference rule in the sense of Definition 3.2:

$$f_{\text{res}} : \mathcal{P}(\mathcal{F}) \rightarrow \mathcal{F}$$

The function $f_{\text{res}}(\{C_1, C_2\})$ is defined if and only if there exists a *unique* propositional variable $p \in V$ such that p occurs as a positive literal in C_1 and $\neg p$ occurs as a negative literal in C_2 (or vice versa). Writing $C_1 = \Delta_1 \vee p$ and $C_2 = \Delta_2 \vee \neg p$, the rule produces the *resolvent* $\Delta_1 \vee \Delta_2$:

$$\frac{\Delta_1 \vee p \quad \Delta_2 \vee \neg p}{\Delta_1 \vee \Delta_2}$$

Treating clauses as sets of their constituent literals, we can also express the resolvent $C_3 = (C_1 \setminus \{p\}) \cup (C_2 \setminus \{\neg p\})$. If both Δ_1 and Δ_2 are empty, the resolvent is the empty clause $\square$. We can always express a clause of one literal as a disjunction between $\perp$ and the literal (logical equivalence follows from S3 and S6) and then the resolvent between the literal and its negation is $\perp$. That also explains why $\perp$ and $\square$ are logically equivalent.

If more than one variable satisfies the above condition (i.e. multiple variables appear with opposite polarity across C_1 and C_2), then every possible resolvent would contain both p_i and $\neg p_i$ for some propositional variable p_i , making it a *tautological clause* (satisfied by every interpretation $v \in \mathcal{I}$, since at least one of L or $\neg L$ always holds).

Proposition 4.8 (Soundness of the Resolution Rule). The resolution rule is sound in the sense of Definition 3.7: for any atomic variable $p \in V$ and any two clauses $C_1 = \Delta_1 \vee p$ and $C_2 = \Delta_2 \vee \neg p$, the resolvent is a logical consequence of the two input clauses:

$$\{C_1, C_2\} \models \Delta_1 \vee \Delta_2$$

Proof. Let $v \in \mathcal{I}$ be an arbitrary interpretation such that $v \models C_1$ and $v \models C_2$. We must show that $v \models \Delta_1 \vee \Delta_2$. Since v is a total boolean valuation (Definition 3.1, Semantics), exactly one of the following cases holds.

Case 1: $v(p) = 1$, i.e. $v \models p$ by (S1). By (S4), $v \not\models \neg p$. Since $v \models C_2 = \Delta_2 \vee \neg p$, rule (S6) ensures that $v \models \Delta_2$ or $v \models \neg p$; as the latter is excluded, we have $v \models \Delta_2$. Applying (S6) again, $v \models \Delta_1 \vee \Delta_2$.

Case 2: $v(p) = 0$, i.e. $v \not\models p$ by (S1). Since $v \models C_1 = \Delta_1 \vee p$, rule (S6) gives that $v \models \Delta_1$ or $v \models p$; as the latter is excluded, we have $v \models \Delta_1$. Applying (S6), $v \models \Delta_1 \vee \Delta_2$.

In both cases $v \models \Delta_1 \vee \Delta_2$. Since v was an arbitrary interpretation satisfying $\{C_1, C_2\}$, we conclude by Definition 3.4 that $\{C_1, C_2\} \models \Delta_1 \vee \Delta_2$. $\square$

Definition 4.9 (Refutation Procedure). Starting from refutation set S , the procedure repeatedly applies f_{res} to any two clauses drawn from S or previously derived resolvents, adding each new resolvent back into the working set. Resolution calculus is refutation-complete (and sound) which ensures $\Gamma \models \phi \iff S \vdash_{D_{\text{res}}} \square$ (Gallier [1986]).

5 Hilbert Calculus

The Hilbert calculus is a proof calculus $D_{\text{hil}} = \langle \Sigma_{A_{\text{hil}}}, R_{\text{hil}} \rangle$ in which we have three axiom schemas and a single inference rule. We present a version of Hilbert Calculus that is complete for formulas with implications and negations. We can convert conjunctions and disjunctions to the desired form using the following rules.

$$\bullet (A \vee B) \equiv (\neg A \rightarrow B) \text{ , } (A \wedge B) \equiv \neg(A \rightarrow \neg B)$$

The axiom schema $\Sigma_{A_{\text{hil}}}$ is a finite set of formula templates in which *placeholder formulas* ϕ, ψ, χ stand for arbitrary formulas in $\mathcal{F}$ (Definition 3.3).

Definition 5.1 (Hilbert Axiom Schemas). $\Sigma_{A_{\text{hil}}}$ consists of the following three schemas:

$$\begin{aligned} \text{(H1)} \quad & (\phi \rightarrow (\psi \rightarrow \phi)) \\ \text{(H2)} \quad & ((\phi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))) \\ \text{(H3)} \quad & ((\neg\psi \rightarrow \neg\phi) \rightarrow (\phi \rightarrow \psi)) \end{aligned}$$

Definition 5.2 (Modus Ponens). The only inference rule of D_{hil} is *modus ponens*. Given $(\phi \rightarrow \psi)$ and ϕ , it derives ψ :

$$\frac{(\phi \rightarrow \psi) \quad \phi}{\psi} \text{ (MP)} \qquad f_{\text{MP}}(\{(\phi \rightarrow \psi), \phi\}) = \psi$$

The rule is defined whenever both $(\phi \rightarrow \psi)$ and ϕ are available in the premise set or derived through axioms or rule application. The output formula is uniquely determined by the two input formulas, making the rule fully deterministic: given $\phi \rightarrow \psi$ and ϕ , the only possible conclusion is ψ .

5.1 Soundness

We now verify that D_{hil} is sound in the sense of Definition 3.7. To do that, we show the axiom schemas are valid and the Modus Ponens rule is sound.

Proposition 5.3 (Validity of the Axiom Schemas). Every instance of (H1),(H2) and (H3) is valid: for any $v \in \mathcal{I}$ and any $\phi, \psi, \chi \in \mathcal{F}$, the instance is satisfied by v .

Proof. We verify each schema by case analysis on the satisfaction relation, using rules (S4) and (S7) from Definition 3.1. From (S7) we get that $v \models (\alpha \rightarrow \beta)$ iff $v \not\models \alpha$ or $v \models \beta$.

(H1): $(\phi \rightarrow (\psi \rightarrow \phi))$. Let $v \in \mathcal{I}$ be an arbitrary interpretation. We do a case split on whether $v \models \phi$ holds.

Case 1: $v \not\models \phi$. By (S7), $(\phi \rightarrow (\psi \rightarrow \phi))$ holds.

Case 2: $v \models \phi$. By (S7), $(\psi \rightarrow \phi)$ holds since its ϕ is satisfied. Applying (S7) again, $(\phi \rightarrow (\psi \rightarrow \phi))$ holds since its $\psi \rightarrow \phi$ is satisfied.

In both cases $v \models (\phi \rightarrow (\psi \rightarrow \phi))$.

(H2): $(\phi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))$. Let $v \in \mathcal{I}$ be arbitrary. We do a case split on whether $v \models \phi$ holds.

Case 1: $v \not\models \phi$. By (S7), $(\phi \rightarrow \chi)$ holds since ϕ is not satisfied. By (S7) again, $((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))$ holds since $(\phi \rightarrow \chi)$ is satisfied. By (S7) again, $((\phi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi)))$ holds since $((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))$ is satisfied.

Case 2: $v \models \phi$. We do a further case split on $v \models \psi$.

Case 2a: $v \not\models \psi$. By (S7), $(\phi \rightarrow \psi)$ does not hold since $v \models \phi$ and $v \not\models \psi$. By (S7), $((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))$ holds since $(\phi \rightarrow \psi)$ is not satisfied. By (S7), $((\phi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi)))$ holds since $((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))$ is satisfied.

Case 2b: $v \models \psi$. We do a final case split on $v \models \chi$.

Case 2b-i: $v \models \chi$. By (S7), $(\phi \rightarrow \chi)$ holds since χ is satisfied. By (S7), $((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))$ holds since $\phi \rightarrow \chi$ is satisfied. By (S7), $((\phi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi)))$ holds since $((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi))$ is satisfied.

Case 2b-ii: $v \not\models \chi$. By (S7), $(\psi \rightarrow \chi)$ does not hold since $v \models \psi$ and $v \not\models \chi$. By (S7), $(\phi \rightarrow (\psi \rightarrow \chi))$ does not hold since $v \models \phi$ and $v \not\models (\psi \rightarrow \chi)$. By (S7), $((\phi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \chi)))$ holds since $(\phi \rightarrow (\psi \rightarrow \chi))$ is not satisfied. $\square$

We can show that H3 is valid using a similar proof with case analysis on ϕ and ψ .

Proposition 5.4 (Soundness of Modus Ponens). The inference rule f_{MP} is sound in the sense of Definition 3.7, i.e. for any $\phi, \psi \in \mathcal{F}$,

$$\{(\phi \rightarrow \psi), \phi\} \models \psi$$

Proof. Let $v \in \mathcal{I}$ satisfy both $(\phi \rightarrow \psi)$ and ϕ . By (S7), $v \models (\phi \rightarrow \psi)$ means $v \not\models \phi$ or $v \models \psi$. Since $v \models \phi$, $v \not\models \phi$ is excluded, so that forces $v \models \psi$. By Definition 3.4, $\{(\phi \rightarrow \psi), \phi\} \models \psi$. $\square$

Theorem 5.5 (Soundness of D_{hil}). The Hilbert calculus is sound: for any $\Gamma \subseteq \mathcal{F}$ and $\phi \in \mathcal{F}$,

$$\Gamma \vdash_{D_{\text{hil}}} \phi \implies \Gamma \models \phi$$

Proof. By Definition 3.7, a proof calculus is sound if its axioms are valid and its inference rules are sound. As per proposition 5.3 all instances of (H1), (H2) and (H3) are valid, and Proposition 5.4 establishes that modus ponens is sound. Therefore D_{hil} is sound. $\square$

Remark 5.6 (Completeness). The Hilbert calculus is complete for formulas with implications and negations (Wasilewska [2018]).

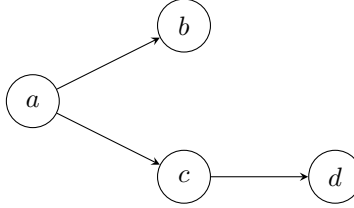
6 Proving Reachability in Directed Graphs

We now formulate reachability in a directed graph as a proposition to be proven in a propositional logic system where given the directed graph, the arcs act as the premise.

Definition 6.1 (Directed Graph). A *directed graph* is a pair $G = (N, E)$ where N is a finite, non-empty set of *nodes* and $E \subseteq N \times N$ is a set of *arcs*. We write $x \rightarrow_G y$ to denote that there is an arc from x to y , i.e. $(x, y) \in E$.

Definition 6.2 (Propositional Reachability Encoding). Let $G = (N, E)$ be a directed graph. For each node $x \in N$, we introduce a distinct propositional variable $R_x \in V$, interpreted as the proposition “node x is reachable”. The premise set $\Gamma_G \subseteq \mathcal{F}$ consists of one formula per arc i.e. $\Gamma_G = \{ (R_x \rightarrow R_y) \mid (x, y) \in E \}$. Each premise encodes the fact that if x is reachable and there is an arc from x to y , then y is reachable. Under this encoding the reachability claim “node t is reachable from s in G ” is expressed as $\Gamma_G \models (R_s \rightarrow R_t)$

Example graph. Let $G = (\{a, b, c, d\}, \{a \rightarrow_G b, a \rightarrow_G c, c \rightarrow_G d\})$:



By Definition 6.2 the premise set is:

$$\Gamma_G = \{ \underbrace{(R_a \rightarrow R_b)}_{C_1}, \underbrace{(R_a \rightarrow R_c)}_{C_2}, \underbrace{(R_c \rightarrow R_d)}_{C_3} \}$$

We wish to prove that d is reachable from a , i.e. $\Gamma_G \models (R_a \rightarrow R_d)$.

6.1 Proof via Hilbert Calculus

We derive $(R_a \rightarrow R_d)$ from Γ_G using the axiom schemas of Definition 5.1 and Modus Ponens (Definition 5.2). We observe that C_1 plays no role, and intuitively it makes sense as we have a dead end at b .

(1)	$(R_c \rightarrow R_d)$	premise C_3
(2)	$((R_c \rightarrow R_d) \rightarrow (R_a \rightarrow (R_c \rightarrow R_d)))$	(H1): with $\phi = (R_c \rightarrow R_d), \psi = R_a$
(3)	$(R_a \rightarrow (R_c \rightarrow R_d))$	$f_{MP}(\{(2), (1)\})$
(4)	$((R_a \rightarrow (R_c \rightarrow R_d)) \rightarrow ((R_a \rightarrow R_c) \rightarrow (R_a \rightarrow R_d)))$	(H2): with $\phi = R_a, \psi = R_c, \chi = R_d$
(5)	$((R_a \rightarrow R_c) \rightarrow (R_a \rightarrow R_d))$	$f_{MP}(\{(4), (3)\})$
(6)	$(R_a \rightarrow R_c)$	premise C_2
(7)	$(R_a \rightarrow R_d)$	$f_{MP}(\{(5), (6)\})$

In the 2nd and 4th step, we have to guess the *right* instantiation of the axiom schema which is impossible for a machine as there can be infinite axioms from any schema (Wasilewska [2018]). Hence, Hilbert calculus is not suitable for automation.

6.2 Proof via Resolution

We apply the refutation procedure mentioned in Definition 4.9 to show that $\Gamma_G \cup \{\neg(R_a \rightarrow R_d)\}$ does not hold under any interpretation i.e. we can derive logical contradiction. The negation of the target is:

$$\neg(R_a \rightarrow R_d) \xrightarrow{\text{Implication Elimination}} \neg(\neg R_a \vee R_d) \xrightarrow{\text{De Morgan, Double Negation}} (R_a \wedge \neg R_d)$$

By Lemma 4.6 this conjunction splits into two unit clauses. Each premise in Γ_G is already an implication and hence can be expressed as a clause through one application of implication elimination (Definition 4.4, rule 1) converts each to a clause. The full refutation set is:

$$S = \{ \underbrace{\{\neg R_a, R_b\}}_{C_1}, \underbrace{\{\neg R_a, R_c\}}_{C_2}, \underbrace{\{\neg R_c, R_d\}}_{C_3}, \underbrace{\{R_a\}}_{N_1}, \underbrace{\{\neg R_d\}}_{N_2} \}$$

where N_1 and N_2 come from the negated target.

We apply the resolution rule from Definition 4.7, which guarantees soundness at each step by Proposition 4.8.

$$\begin{array}{llll} \text{(R1)} & \{R_c\} & \text{resolve } C_2 \text{ and } N_1 \text{ on } R_a & f_{\text{res}}(\{C_2, N_1\}) \\ \text{(R2)} & \{R_d\} & \text{resolve } C_3 \text{ and (R1) on } R_c & f_{\text{res}}(\{C_3, (R1)\}) \\ \text{(R3)} & \square & \text{resolve (R2) and } N_2 \text{ on } R_d & f_{\text{res}}(\{(R2), N_2\}) \quad R_d \text{ and } \neg R_d \text{ contradict} \end{array}$$

The empty clause $\square$ is derived, so by Definition 4.9 we conclude $\Gamma_G \vdash (R_a \rightarrow R_d)$. Here, we started with a finite set of clauses and in general the number of unique clauses in a propositional logic system is finite. As a result, resolution always terminates for propositional logic. We are either able to derive the empty clause or we are no longer able to add new clauses (Gallier [1986]).

7 Conclusion

In this paper, we formalized propositional logic as a 4-tuple system $\mathcal{L} = \langle \Sigma, \mathcal{F}, M, D \rangle$ and studied two proof calculi. We defined inference rule, axiom schema, syntactic derivability, formal proof, soundness, and completeness, and proved that both the Resolution and Hilbert calculi are sound for propositional logic. The Resolution calculus is well-suited for automation because the procedure is guaranteed to terminate: either the empty clause is derived (confirming $\Gamma \models \phi$), or no new resolvents can be generated (confirming $\Gamma \not\models \phi$ by refutation-completeness). The Hilbert calculus, by contrast has an infinite supply of axiom instances which means the search space is unbounded. Finally, we demonstrated both calculi on a concrete example by encoding graph reachability as a propositional logic formula and generating proofs via both calculi.

References

- John Corcoran and Idris Samawi Hamid. Schema. In *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Fall 2022 edition, 2022.
- Herbert B. Enderton. *A Mathematical Introduction to Logic*. Academic Press, 2nd edition, 2001. doi: 10.1016/C2009-0-22107-6.
- Melvin Fitting. *First-Order Logic and Automated Theorem Proving*. Texts in Computer Science. Springer, 2nd edition, 1996. doi: 10.1007/978-1-4612-2360-3.
- Jean H. Gallier. *Logic for Computer Science: Foundations of Automatic Theorem Proving*. HarperCollins Publishers, 1986. doi: 10.2307/2275035.
- Anita Wasilewska. *Logics for Computer Science: Classical and Non-Classical*. Springer Nature Switzerland AG, 2018. doi: 10.1007/978-3-319-92591-2.